HCLTech AMPLIFIED ROUND 2
SOLUTION DOCUMENTATION

LearnPath

AI-Powered Personalized Learning Path Recommender

"Tell us your goal. We'll find your gaps and build the path - not someone else's."

Team submission - github.com/Jathin-24/LearnPath

1. Problem Understanding & Solution Design

- Online learning platforms recommend individual courses, but learners struggle to identify the right **sequence** of resources to reach a specific goal.
- Different learners have different skill levels, interests, career aspirations and learning preferences - a one-size-fits-all approach is ineffective.
- Learners waste hours covering concepts they already know and miss the prerequisites they actually need.

LearnPath addresses this with a closed-loop adaptive system: conversational intake -> profiling -> deterministic assessment -> personalized sequenced roadmap -> learning -> reassessment -> adaptation. The design explicitly rejects "an LLM generates a roadmap" in favor of **an adaptive learning operating system that continuously decides what to learn next** based on evidence.

Direct mapping to the brief: conversational interface (Profiler Agent), learner profiling engine (structured LearnerProfile), recommendation engine (two-path RAG + LLM selection), personalized path generator with prerequisites & milestones, an AI explainer ("why this recommendation"), and a progress dashboard with next recommended actions.

2. The Two-Path Model (Deep Dive)

The recommendation engine is a **hybrid two-path model** that blends dataset grounding with live web expertise. It is the core intellectual contribution of the project.

Aspect	Path A - Dataset	Path B - Open Web
Source of truth	Enriched 80-course, 109,776-review dataset (static, curated)	Live Tavily web + YouTube search (current, broad)
Candidates	TF-IDF + FAISS nearest-neighbour retrieval (k=15)	Tavily top-4 web + top-3 YouTube results
Selection	LLM picks semantic fit from retrieved candidates	LLM synthesizes notes & plans topics (3-6)
Ordering	Prerequisite graph + topological sort (Kahn's algorithm)	Sequential chain (each node prereqs the prior)
Grounding	Course name, summary, concepts, difficulty, review_count	Real URLs with relevance score, freshness, reason
Env. footprint	Free, offline, no model download (free-tier memory)	Requires TAVILY_API_KEY
Best for	Goals covered by the 80-course catalog	Novel / emerging / niche goals

2.1 Routing & confidence filter

The decision of "which path" is made inside `run_path_a()` in `backend/agents/path_a.py`, governed by three constants:

```
SEED_K = 15 # top-N FAISS candidates retrieved
MIN_SIMILARITY_SCORE = 0.35 # cosine-sim threshold a candidate must pass
TEMPLATE_SIMILARITY_THRESHOLD = 0.85 # cached-roadmap reuse threshold
```

- **Query build** - the learner's goal plus every detected skill gap joined with ". " (fallback: "general programming fundamentals").
- **Template cache check** - the query is embedded and matched against prior learners' roadmaps; at cosine >= 0.85 a very similar prior roadmap is reused wholesale (nodes re-locked), skipping all retrieval/LLM work.
- **Retrieve + filter** - 15 candidate courses are retrieved; only those scoring >= 0.35 survive.
- **Whole-goal fallback to Path B** - if **zero** candidates pass 0.35, the module logs `dataset_match_too_weak` and routes the entire roadmap to Path B.

> The 0.35 threshold is evidence-based: live testing on "become a backend developer" showed real matches scoring 0.36-0.59 while noise (Android App Development, React Native, even Ethical Hacking Basics) sat at 0.28-0.32 - no boundary a human would trust from raw vector math, so the LLM refines selection on top.

Crucially, **both paths can be combined in one roadmap**: when a selected Path-A course lists *external prerequisite concepts* (knowledge outside the catalog, e.g. "HTTP Basics"), those become Path-B stub nodes embedded in the same roadmap and are wired as real prerequisite edges. The result is a dataset-grounded roadmap whose external gaps are filled from the open web.

3. Path A - Dataset-Grounded Agent (Deep Dive)

File: `backend/agents/path_a.py`. Responsibility: *RAG retrieval + LLM planning + prerequisite traversal over the 80-course dataset - dataset-grounded roadmap nodes.*

Step 1 - Retrieve candidates

- The goal+gap query is embedded with the TF-IDF vectorizer (4096 features) and searched against the FAISS inner-product index (L2-normalized => cosine similarity).
- **k = SEED_K = 15** candidate courses are returned, score-sorted; those below the 0.35 confidence floor are dropped.

Step 2 - LLM semantic planning

- Each surviving candidate is sent to the LLM as `{course_name, difficulty, concepts[:8], summary}` together with the learner profile (goal, timeline, interests, optional roadmap_instructions).
- The prompt explicitly instructs: *"Exclude anything off-topic... It's fine to select fewer courses than were given - never pad the list."*
- Output is validated against `RoadmapPlanOutput`; names outside the candidate set are filtered; at least one valid name is required; on failure it retries once with a stricter prompt, then fails loud (never silently wrong).
- **This is the semantic-fitness step** - TF-IDF is lexical, so the LLM supplies the semantic judgement about genuine relevance.

Step 3 - Prerequisite graph traversal

- **Closure** - an iterative stack walk expands each selected course's `internal_prerequisites` into the full prerequisite closure set.
- **Topological sort (Kahn's algorithm)** - in-degree counting with a sorted zero-in-degree queue (alphabetical tie-break) guarantees prerequisites appear before dependents; a cycle raises a clear `ValueError`.

Step 4 - Node construction & external promotion

- Each node carries `node_id` (slugified course name), topic, path_type = PATH_A_DATASET, course_name, course_search_link, course_summary, internal_prerequisites (slugified, only if in the selected set), external_prerequisite_concepts, and the first 5 concept tags.
- External prerequisite concepts hit by selected courses are **promoted into Path-B stub nodes** placed first in the roadmap and added to each dependent's prerequisite list - turning dead links into live, fillable web nodes.

4. Path B - Open-Web Agent (Deep Dive)

File: `backend/agents/path_b.py`. Responsibility: *Tavily web/YouTube search + LLM synthesis - open-web roadmap nodes, external-prerequisite fills, and "find more resources" refresh.*

```
QUESTIONS_PER_NODE = 3
MAX_WEB_RESULTS = 4
MAX_YOUTUBE_RESULTS = 3
```

Search

- Web: `"{topic} tutorial guide"`, top 4 results.
- YouTube: `"{topic} tutorial"`, top 3 results, domains restricted to `youtube.com` / `youtu.be`.

Synthesis

- Each result's title/URL + first 800 chars are fed to an LLM (`max_tokens=700`) to produce a 200-400 word plain-prose study guide; degrades gracefully to "use general knowledge, note that clearly" if no results exist.
- Every resource records provenance: `source_type` (web/youtube), a 220-char snippet, Tavily score (3 decimals), published date, and an auto-generated *reason* - e.g. "Top result for this topic from {domain}; published {freshness}; Tavily relevance {score:.0%}."

Three fill shapes

Shape	Trigger	What it does
_fill_resources_only	Brand-new stub, no notes yet	Search + synthesize notes; project/quiz deferred
_supplement_node	Node already has resources ("Find more")	Refresh notes/sources/links only; never touches project/quiz
_fill_node	force=True (explicit Regenerate)	Resources AND project/quiz together

Standalone mode (whole-goal Path B)

- For goals with no dataset coverage, the agent searches `"{goal} roadmap topics to learn in order"`, then an LLM lists 3-6 sequential topics.
- Bare stub nodes are chained (each prereqs the previous), then filled later by the Roadmap Generator's per-node loop calling back into `run_path_b(node_id=...)`.

5. The Dataset & Data Pipeline (Deep Dive)

5.1 The raw dataset

Raw dataset: 109,776 Udemy reviews across 80 courses, with no metadata. Each row is a review tied to a course name (`course_reviews.csv`: Index, Reviews, Course). The metadata - concepts, difficulty, strengths, weaknesses - had to be *engineered*, not assumed.

5.2 One-time enrichment (`enrich_courses.py`)

- Reviews are grouped by course; at most **10 reviews per course** are sampled (`MAX_REVIEWS_PER_COURSE_SAMPLE`).
- **One LLM call per course** (`max_tokens=1000`) extracts: *concepts* (5-12 specific technical concepts), *difficulty* (beginner|intermediate|advanced), *strengths*, *weaknesses*, and a one-sentence *summary*. Parse failures are recorded rather than crashing.
- A **search_link** is built per course: `udemy.com/courses/search/?q={course_name}`.
- `review_count` = number of raw reviews for that course.
- One additional LLM call across the full list (`max_tokens=4000`) infers the **prerequisite DAG** (course -> prerequisite courses), guided by concept overlap and difficulty progression; the prompt nudges most courses toward 0-2 prerequisites.
- **Total cost: ~81 LLM calls, run once.**

5.3 Internal vs external split (`split_prerequisites.py`)

- For each course: `internal_prerequisites` = prerequisites that exist as courses in the catalog; `external_prerequisite_concepts` = those that don't.
- External concepts are exactly what Path A promotes to Path-B stub nodes.

5.4 The validated output (`enriched_courses.json`)

Verified profile of the enriched dataset:

- **80 courses**; total **109,776 reviews**.
- **106 internal prerequisite edges** and **13 external prerequisite concepts** (across 10 courses).
- Difficulty mix: **56 intermediate / 18 beginner / 6 advanced**.
- **0 parse errors**; 80/80 have summary + search_link; 0 empty concept lists; the assertion test verifies every internal_prerequisites reference resolves to a real node earlier in the roadmap (0 dangling edges).


```
"React Native Mobile Development": {
"concepts": ["JSX components", "StyleSheet", "Flexbox layout", "..."],
"difficulty": "intermediate",
"strengths": "Reviewers consistently praise the practical, hands-on assignments...",
"weaknesses": "They repeatedly note a desire for more complex projects...",
"summary": "Overall, the course is seen as a solid, value-for-money...",
"search_link":
"https://www.udemy.com/courses/search/?q=React+Native+Mobile+Development",
"review_count": 1402,
"internal_prerequisites": ["JavaScript Fundamentals", "React.js Development"],
"external_prerequisite_concepts": []
}
```


6. RAG Index & Retrieval (Deep Dive)

6.1 Index build (`backend/rag/build_index.py`)

- Each course's embedding text = `"{course_name}. Difficulty: {difficulty}. Concepts: {concepts}. {summary}"`.
- Vectorizer: `TfidfVectorizer(max_features=4096, sublinear_tf=True, norm="l2")` - **4096 features**.
- Embeddings (dense float32) -> `faiss.IndexFlatIP` (flat inner-product). Because vectors are L2-normalized, inner product = **cosine similarity**.
- Artifacts committed to the repo: `course_index.faiss`, `course_metadata.json` (80 names), `tfidf_vectorizer.pkl`.

6.2 Query-time retrieval (`backend/rag/retriever.py`)

- `retrieve(query, k)` loads cached index/vectorizer/courses, transforms and L2-normalizes the query, runs `index.search`, and returns the <=k results with enriched fields, descending by cosine score.
- `embed_text()` returns the 4096-dim vector, reused by Path A for the template-similarity cache.
- A clear `FileNotFoundError` guides the operator if the index is missing.

6.3 Why the TF-IDF approach?

Render free-tier constraint (the deciding factor): the earlier stack bundled torch + sentence-transformers, which blew past the free tier's build/startup memory budget. TF-IDF has no model download and no heavy ML dependency, so the same 80-course index builds, loads and serves comfortably on Render's 512MB free tier. The accepted tradeoff - TF-IDF is lexical - is compensated by the hybrid planner letting the LLM do semantic candidate selection on top of retrieval. No paid embedding API anywhere.

7. Multi-Agent Orchestration & Roadmap Flow (Deep Dive)

Orchestrated by LangGraph (`backend/orchestrator/graph.py`). The compiled build chain is **PROFILER -> ASSESSMENT -> PATH_A -> ROADMAP_GENERATOR -> END**, with two routing functions: `route_entry` trusts `next_agent` to pick where a resumed session starts, and `route_next` checks `awaiting_input` first (so a paused node routes straight to END, preventing infinite self-loops) then cascades to `next_agent`.

The 8 agents (backend/agents/)

Agent	File	Responsibility
Profiler	profiler.py	Conversational onboarding intake -> updates LearnerProfile (goal, timeline, interests, skills) in the same LLM call as the reply.
Assessment	assessment.py	Two-phase skill-gap check - concept checklist + adaptive MCQ -> SkillGapMap; grading is deterministic, never LLM-as-judge.
Path-A	path_a.py	RAG retrieval + 0.35 confidence filter + LLM planning + prerequisite-graph traversal -> dataset-grounded nodes (falls back to Path B if nothing fits).
Path-B	path_b.py	Tavily web/YouTube search + LLM synthesis -> open-web nodes, external-prerequisite fills, "find more resources" refresh.
Roadmap Generator	roadmap_generator.py	Merges Path-A/B output into the final Roadmap, fills web-node resources eagerly, owns template-cache writes and lazy project/quiz/subtopic generation.
Knowledge Extractor	knowledge_extractor.py	Turns imported context / resume text into structured knowledge-base facts (LLM path with regex fallback for resumes).
Explainer	explainer.py	On-demand "why is this topic in my roadmap?" grounded in profile data + prerequisite-graph position (/roadmap/explain/{node_id}).
Tutor	tutor.py	Mid-roadmap Topic Tutor answering questions grounded in the current node (/chat).

Generate -> review -> confirm

- **Generate** - Roadmap Generator splits nodes into dataset vs web; eagerly fills every PATH_B_OPEN_WEB stub's resources; sets stage = ROADMAP_REVIEW and pauses the graph for user review. It also writes a skeleton to the `roadmap_templates` cache (goal_text, TF-IDF goal_embedding, nodes) so a similar future goal can reuse it - skeleton only, never project/quiz content.
- **Review** - the learner can reorder (locked nodes), skip, add, modify (re-runs Path-A/B with roadmap_instructions) or delete before committing.
- **Confirm** - `POST /roadmap/confirm` sets stage = IN_PROGRESS and unlocks **exactly one node at a time** in topological order; Path-B nodes without notes are never unlocked.

Lazy generation (don't spend LLM calls on unreached modules)

- `ensure_subtopics` -> 4-8 sub-concepts per node, first AVAILABLE, rest LOCKED.
- `generate_subtopic_quiz` -> 2-question quiz per sub-concept on "Done Learning".
- `generate_final_content` -> ONE combined LLM call per node for a ProjectAssignment (title, description, success criteria) + TopicAssessment (3 questions) + estimated_days, fired only when every subtopic is PASSED/SKIPPED. Path A uses `_generate_node_content`; Path B uses `generate_project_and_quiz_from_notes` (no re-search).

8. LLM Client & Failover (Deep Dive)

```
LLM_PROVIDERS=groq,groq,openrouter,xai # priority order, comma-separated
LLM_API_KEYS=key1,key2,key3,key4 # position-matched
LLM_MODELS=model1,model2,model3,model4 # one per endpoint
DATABASE_URL=postgresql://localhost/learning_path_db
TAVILY_API_KEY=...
```

- Priority-ordered endpoints; lengths of providers/keys/models must be equal (validated at load).
- Endpoint states: **HEALTHY**, **COOLING_DOWN** (rate-limited, skipped until cooldown), **DEAD** (non-retryable, skipped for the run); cooling endpoints auto-re-promote.
- Error classification: 429/"rate limit"/"quota" -> cooling 60s; 401/403/"invalid api key" -> DEAD; >=500/"timeout"/"connection" -> transient retry same endpoint with 1.0*2^attempt backoff (max 2), then next endpoint.
- All endpoints exhausted -> `AllEndpointsExhaustedError` caught by the API, which returns a friendly 502 instead of crashing.
- Providers: xai, groq, openrouter (OpenAI-compatible) + native anthropic. `status()` exposes a key-redacting /health view (label = provider:model:...KEY_LAST4).

8b. System Architecture & Stack

Runtime flow: React/Vite -> FastAPI -> LangGraph -> RAG (FAISS + TF-IDF) -> multi-provider LLM / Tavily -> PostgreSQL.

The system is one request pipeline: the browser talks to FastAPI routes (thin wrappers only), which drive a LangGraph state machine of eight agents. Agents read/write one shared Pydantic `AppState` (persisted as Postgres JSONB per session), retrieve courses via the local TF-IDF+FAISS RAG index, and call out to the multi-provider LLM client or Tavily only when a step needs external intelligence. Responses are validated against schemas before ever reaching state.

Layer	Technology	Role in the flow
Frontend	React 19 + TypeScript + Vite + Tailwind v4	Renders onboarding, roadmap graph (React Flow), analytics (Recharts), auth, dashboard
API	FastAPI + Pydantic v2	Thin route wrappers -> orchestration/db/rag; auto docs at /docs
Orchestrator	LangGraph	PROFILER -> ASSESSMENT -> PATH_A -> ROADMAP_GENERATOR -> END; resume/route logic
Agents	8 specialized agents (profiler, assessment, path_a, path_b, roadmap_generator, knowledge_extractor, explainer, tutor)	Each owns one step of the learning loop
State	PostgreSQL JSONB (no ORM), tables auto-created	Per-session AppState, users, knowledge base, resume files
RAG	scikit-learn TF-IDF + FAISS (committed index)	Top-k course retrieval grounding Path A

Layer	Technology	Role in the flow
External AI	Multi-provider LLM (Groq/OpenRouter/xAI) + Tavily	Generation, grading prompts, web/YouTube resource search

- **Frontend:** React 19, TypeScript, Vite, Tailwind v4; React Flow (roadmap) + Recharts (analytics).
- **Backend:** Python 3.12, FastAPI, Pydantic v2, LangGraph; eight agents; PostgreSQL JSONB per session (no ORM); tables auto-created.
- **RAG:** scikit-learn TF-IDF + FAISS over the enriched 80-course dataset.
- **LLM:** multi-provider (Groq + OpenRouter + xAI) with automatic failover; Tavily for web search.
- **Data pipeline:** enrich_courses.py (one-time LLM pass) + split_prerequisites.py -> validated enriched_courses.json (0 parse errors).

9. Local Setup & Execution

Prerequisites: Python 3.11+, Node.js 18+, PostgreSQL running locally (or a Supabase URL).

Backend (from the repo root):

- python -m venv .venv and activate it (.venv\Scripts\activate on Windows)
- pip install -r requirements.txt
- copy .env.example .env -> fill LLM_PROVIDERS / LLM_API_KEYS / LLM_MODELS / DATABASE_URL / TAVILY_API_KEY (ACCESS_TOKEN_SECRET optional in dev)
- Create the database: CREATE DATABASE learning_path_db; (tables auto-create on first run)
- (Optional) rebuild RAG index: python -m backend.rag.build_index - only if enriched_courses.json changes
- Run: uvicorn backend.api.main:app --port 8000

Frontend:

- cd frontend
- npm install
- npm run dev - open http://localhost:5173

The frontend works out of the box against http://127.0.0.1:8000. Only if your backend lives elsewhere, set VITE_API_BASE_URL in frontend/.env.

Health/status: http://127.0.0.1:8000/health - API docs: http://127.0.0.1:8000/docs

Windows note: do not use --reload - the WatchFiles reloader orphans the worker process. Restart manually.

10. Testing, Verification & Benchmark

- **Backend:** pytest backend/tests/ -v (60+ tests, real Postgres + real LLM, no mocking).
- **AI benchmark:** python -m backend.benchmarks.ai_benchmark --limit 5 (--resume after rate limits; --offline for deterministic heuristics). The hybrid engine is scored against a generic-LLM baseline and a RAG-only baseline over 30 learner profiles across 5 quality dimensions (relevance, ordering, personalization, feasibility, grounding).
- **Frontend:** npm run lint (oxlint) and npx tsc --noEmit.

- **Reliability:** deterministic grading, append-only progress log, multi-key LLM failover, validated LLM output, 0 dangling prerequisite edges.
- **Security:** bcrypt password hashing; signed access tokens (ACCESS_TOKEN_SECRET) enforce session/resource ownership; Guest flow stays open.

11. Deployment (if available)

Piece	Host	Notes
Database	Supabase	Session pooler URL; tables auto-create on startup
Backend	Render	uvicorn backend.api.main:app --host 0.0.0.0 --port $PORT
Frontend	Vercel	Root directory: frontend; framework Vite

Full configuration (env vars, CORS via ALLOWED_ORIGINS, wake-up notes) is in docs/deployment_guide.md. RAG artifacts are committed - no build step at deploy.

12. Challenges Faced & How We Solved Them

The biggest risks were infrastructure and data-quality, not features. Each is listed with the concrete fix that shipped.

Challenge	How we solved it
Raw dataset had no metadata - 109,776 bare reviews across 80 courses	One-time enrichment pass: sampled at most 10 reviews per course, one LLM call per course to extract concepts, difficulty, strengths, weaknesses and summary; one full-list call inferred the prerequisite DAG (~81 calls total, run once). Verified: 0 parse errors.
Render free tier limits memory to 512MB; torch + sentence-transformers blew the budget	Moved to TF-IDF (4096 features) + FAISS flat inner-product. The index is committed to the repo - no model download, no heavyweight ML build step on the server.
Render ran Python 3.14, where pydantic-core has no wheel - the Rust/maturin compile crashed the build	Pinned the runtime to Python 3.12.11: added .python-version (the legacy runtime.txt is now ignored by Render) and set PYTHON_VERSION=3.12.11 in the dashboard, which takes highest precedence.
LLM providers are flaky and rate-limited	Priority-ordered multi-provider client (Groq / OpenRouter / xAI): HEALTHY / COOLING_DOWN / DEAD state machine, 60s cooldown on 429, exponential backoff on timeouts, auto-failover to the next endpoint; friendly 502 instead of a crash when all are exhausted.
TF-IDF is lexical - vector similarity is not semantic fit (noise sat at 0.28-0.32, real matches at 0.36-0.59)	Evidence-based 0.35 confidence floor plus a whole-goal fallback to the open web. The hybrid planner keeps retrieval cheap but lets the LLM make semantic selection on top - the best of both worlds with no paid embedding API.
LLMs can fabricate course names or topics	Every agent's output is validated against the retrieved candidate set and the shared Pydantic schema; one strict-prompt retry happens before the agent fails loud - it is never silently wrong.
Selected courses often had prerequisites outside the catalog	split_prerequisites.py separates internal (in-catalog) from external edges; external concepts are promoted to Path-B stub nodes embedded in the same roadmap, so there are no dead links.

Challenge	How we solved it
Production wiring bugs surfaced (a 500 on resume upload; a stray carriage-return in the Vercel env broke the inlined API URL)	The upload 500 was a missing Request parameter - fixed and covered by tests. The env bug was caught by inspecting the deployed JS bundle and fixed by re-adding the variable from a clean LF-only value file.

Takeaway: the free tier forced real engineering tradeoffs (TF-IDF over embeddings, pinned runtimes, multi-provider failover) - each one made the system lighter, cheaper and more robust.

Appendix - Did we build what was asked?

The HCLTech AMPLIFIED brief asked for a conversational interface, learner profiling, a recommendation engine, a personalized path generator with prerequisites and milestones, an AI explainer, and a progress dashboard. Every one of these is implemented and integrated, and we went beyond with adaptation (roadmap regeneration), per-topic quizzes and projects, resource provenance, engagement features, and an AI quality benchmark.

Brief requirement	Built?	Where
Conversational interface (goal in natural language)	Yes	Profiler Agent intake + Chat
Learner profiling engine	Yes	LearnerProfile, resume/context import
Recommendation engine (courses / projects / resources)	Yes	Two-path RAG + LLM + Tavily
Personalized path generator w/ prerequisites & milestones	Yes	Roadmap Generator, topological order
AI assistant - explains each recommendation	Yes	Explainer Agent + Topic Tutor
Dashboard - progress, skills, milestones, next actions	Yes	Dashboard + Analytics
Adapt suggestions based on feedback/progress	Yes	Roadmap regeneration, benchmark